



Absolutely 😊 We'll do the DevTools experiment **after you build the Next.js app** so you can actually see the HTML, RSC requests, navigation, and caching instead of just imagining them. 



Next → Lesson 6.8

We'll move into:

Hydration vs Reconciliation vs RSC Updates

We'll answer:

Show more ▾



Next.js Lesson 6.8 — Hydration vs Reconciliation vs RSC Updates

Yes! Let's continue exactly from here. 😊

This is an **important mental-model lesson** because these three things are often mixed together:

- **Hydration** → making server-rendered HTML interactive
- **Reconciliation** → React figuring out what UI needs to change
- **RSC update** → Next.js fetching new Server Component output from the server

We'll use **one example** all the way through.

1. Our example

Imagine this Next.js page:

 TypeScript 

```
// app/products/page.tsx
```

```
export default async function ProductsPage() {  
  const products = await getProducts();  
  
  return (  

```

```
<main>
  <h1>Products</h1>

  <ProductList products={products} />

  <AddToCartButton />
</main>
);
}
```

And:

```
</> TypeScript
// AddToCartButton.tsx

"use client";

export function AddToCartButton() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Cart: {count}
    </button>
  );
}
```

So we have:

```
ProductsPage
├── <h1>Products</h1>      ← Server Component
├── ProductList            ← Server Component
└── AddToCartButton        ← Client Component
```

Now let's see what happens.

2. Initial request

Browser requests:

```
/products
↓
Next.js server
```

Next.js executes the Server Component:

```
</> TypeScript  
ProductsPage()
```



It gets the products:

```
Database/API  
  ↓  
products
```



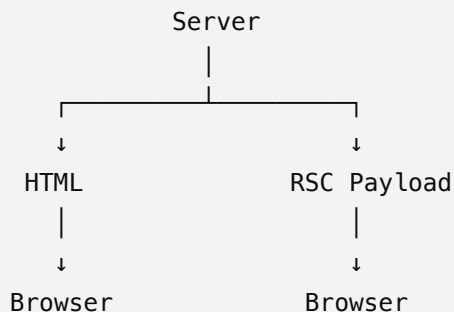
Then React renders the Server Component tree.

Conceptually:

```
ProductsPage  
  ↓  
Server Component tree  
  ↓  
React renders it
```



There are **two important outputs** involved:



3. What is the HTML for?

The HTML gives the browser something it can **display immediately**.

For example:

```
HTML  
  
<main>  
  <h1>Products</h1>  
  
  <div>iPhone</div>  
  <div>MacBook</div>
```



```
<button>Cart: 0</button>
</main>
```

The browser can paint this.

So:

```
Server
  ↓
HTML
  ↓
Browser
  ↓
User sees page
```



But here's the interesting part.

The browser currently has:

```
HTML
<button>Cart: 0</button>
```



But clicking it won't magically execute:

```
<\/ TypeScript
setCount(count + 1)
```



because that JavaScript hasn't been connected yet.

That's where **hydration** comes in.

4. What exactly is hydration? 🧠

Hydration is basically:

React takes the existing server-rendered HTML and attaches React behavior to the Client Components.

Notice the important distinction:

Server Component

```
<\/ TypeScript
```



```
<h1>Products</h1>
```

doesn't need hydration.

Why?

Because it isn't interactive in the browser.

Client Component

```
</> TypeScript
```

```
"use client"
```

```
<button onClick={...}>
```

needs JavaScript in the browser.

So conceptually:

```
HTML already exists
```

```
↓
```

```
React loads Client Component JS
```

```
↓
```

```
React identifies:
```

```
"This button belongs to AddToCartButton"
```

```
↓
```

```
React hydrates it
```

```
↓
```

```
onClick becomes functional
```

5. Does React hydrate the entire page?

No.

This is a BIG point.

Suppose:

```
ProductsPage
```

```
|
```

```
├─ h1
```

```
Server
```

```
|
```

```
├─ ProductList
```

```
Server
```

```
|
```

```
└─ AddToCartButton
```

```
Client
```

Hydration isn't:

Hydrate EVERYTHING ❌



Instead, the Client Component boundary determines where browser-side React behavior is needed.

Conceptually:

Server-rendered UI



```
|
├── Server Component
│   └── no hydration
|
├── Server Component
│   └── no hydration
|
└── Client Component
    └── hydrate / run client React
```

That's one of the major benefits of the RSC architecture.

6. Now comes reconciliation

Hydration and reconciliation are **not the same thing**.

Think of reconciliation as:

React comparing the previous React UI representation with the new one and determining what needs to change.

Example:

Initially:

Cart: 0



User clicks:

Add to cart



State changes:

</> TypeScript



```
setCount(1)
```

React now renders the Client Component again:

```
</> TypeScript
```

```
<button>  
  Cart: 1  
</button>
```

React compares:

```
Previous:  
<button>Cart: 0</button>
```

```
New:  
<button>Cart: 1</button>
```

It determines:

```
Only text changed:  
0 → 1
```

Then React updates the DOM accordingly.

That's **reconciliation**.

7. Hydration vs reconciliation

Here's the clean distinction:

Concept	What is happening?
Hydration	Connect React behavior to existing server-rendered HTML
Reconciliation	Compare previous UI with new UI and determine updates
DOM update	Actually modify browser DOM

Think:

```
INITIAL PAGE
```

```
HTML already exists
```

```
↓
```

```
Hydration
```

↓
React becomes interactive

Then later:

STATE/PROPS CHANGE

↓
React renders new UI
↓
Reconciliation
↓
DOM update



8. Now the interesting part: RSC Updates

Let's say we have:

```
</> TypeScript  
  
router.refresh()
```



What happens?

A lot of people incorrectly think:

```
refresh()  
↓  
Browser reload  
↓  
HTML downloaded again
```



✗ Not exactly.

`router.refresh()` asks Next.js to **re-render the Server Components for the current route** and send the updated RSC result to the browser.

Conceptually:

```
router.refresh()  
↓  
Request to Next.js  
↓  
Server Components execute again  
↓  
New RSC Payload  
↓
```



Browser



React merges the result into existing UI

9. Example with router.refresh()

Suppose the server initially returns:

Products:

Apple
Samsung
Google



Then something changes on the server/database:

Apple
Samsung
Google
OnePlus ← new



Client executes:

```
</> TypeScript  
router.refresh();
```



Next.js asks the server to render the Server Component again.

Server now produces:

Products:

Apple
Samsung
Google
OnePlus



The browser receives the **RSC update**.

Then React uses that new Server Component result to update the existing React tree.

So:

```
router.refresh()  
  ↓  
Server re-render  
  ↓  
RSC Payload  
  ↓  
React receives new Server Component tree  
  ↓  
Reconciliation  
  ↓  
DOM changes
```



Notice:

RSC update and reconciliation are different steps.

10. Does router.refresh() re-hydrate everything?

No.

This is another important interview point.

It doesn't mean:

```
Destroy entire React application  
  ↓  
Download everything  
  ↓  
Hydrate everything again
```



Instead, Next.js receives updated Server Component output and React integrates it into the existing client-side tree.

Existing Client Component state can be preserved when the component remains in the same position/tree structure.

For example:

```
Cart: 5
```



You call:

```
</> TypeScript  
  
router.refresh()
```



The server updates the product list.

You don't necessarily lose:

Cart: 5



because the Client Component isn't simply being treated as a brand-new browser page.

11. Now `<Link>` navigation

Suppose we're on:

/products



and click:

`</>` TypeScript



```
<Link href="/profile">
  Profile
</Link>
```

This is different from a normal:

 HTML



```
<a href="/profile">
```

With App Router client-side navigation, Next.js can request the RSC representation for the destination.

Conceptually:

```
/products
↓
click <Link>
↓
Next.js navigation request
↓
Server renders /profile Server Components
↓
RSC Payload
↓
Browser
↓
React integrates new tree
```



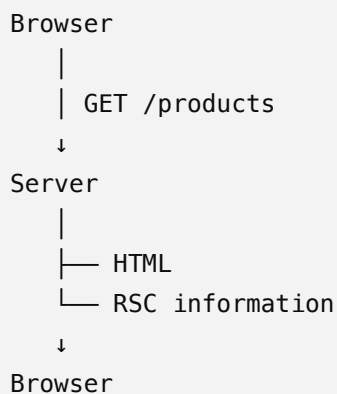
↓
UI becomes /profile

The browser doesn't necessarily need to perform a complete document reload.

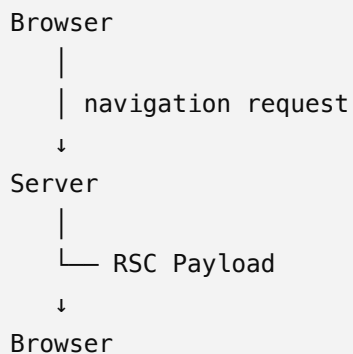
12. So what is actually travelling over the network?

This is the beautiful part. 😊

For a full initial request:



For a client-side navigation:



The existing document doesn't necessarily get replaced with a completely new HTML document.

React uses the RSC information to update the UI.

13. Three scenarios side-by-side

This is the mental model I want you to remember:

● Initial page load

Request
↓
Server
↓
HTML + RSC
↓
Browser displays HTML
↓
Client Components hydrate



Client state update

Example:

`</> TypeScript`

```
setCount(1)
```



User interaction
↓
Client Component state changes
↓
React renders
↓
Reconciliation
↓
DOM update



No server request required.

router.refresh()

router.refresh()
↓
Server Component render
↓
RSC Payload
↓
React receives updated server tree
↓
Reconciliation
↓
Existing UI updated



14. And `<Link>` navigation?

```
<Link href="/profile">
```

↓

Client-side navigation

↓

Server renders destination RSC tree

↓

RSC Payload

↓

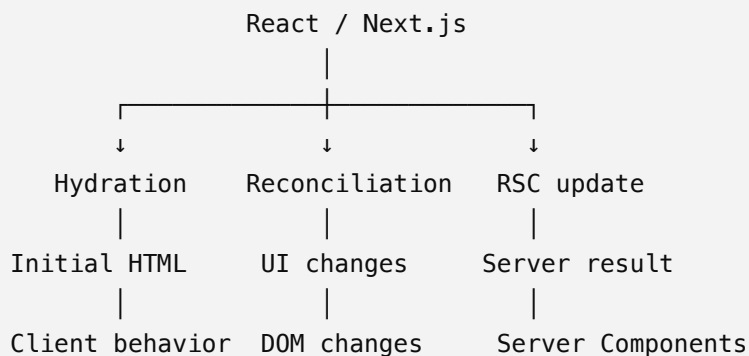
React updates tree

↓

Profile UI



So you can think:



15. The most important distinction 🔥

Imagine your interviewer asks:

"What's the difference between hydration and reconciliation in Next.js?"

You can say:

Hydration happens after the initial server render, where React attaches client-side behavior to the server-rendered HTML for Client Components.

Reconciliation is the ongoing React process of comparing the previous and new UI trees to determine what needs to change.

Then if they ask:

"Where do RSCs fit?"

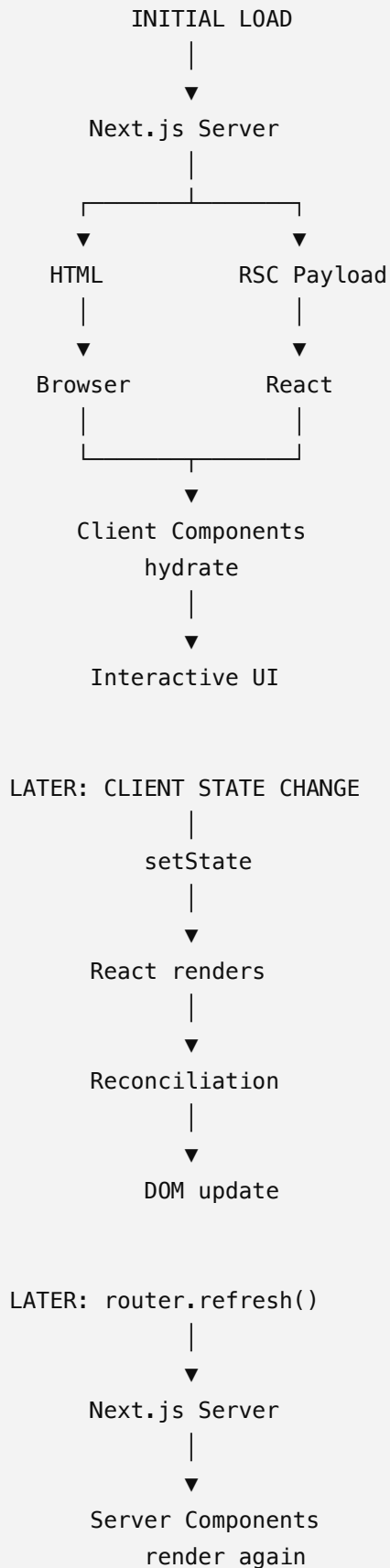
Say:

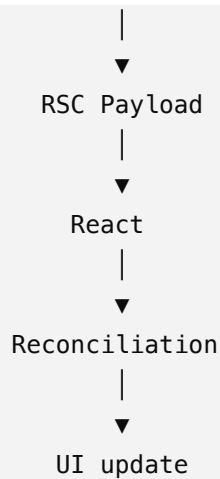
Server Components execute on the server and produce an RSC payload. During navigation or `router.refresh()`, Next.js can send updated RSC payloads to the browser, and React integrates those results into the existing UI through its rendering and reconciliation process.

That's a very solid answer. 🎯

One final mental picture

Keep this in your head:





🧠 The three words

Hydration = "make existing HTML interactive."

Reconciliation = "figure out what UI changed."

RSC update = "get updated Server Component output from the server."

And importantly:

RSC update can lead to reconciliation, but RSC update itself is not reconciliation.

That distinction is *very* interview-worthy. 🚀



The existing document doesn't necessarily get replaced with a completely new HTML document.

React uses the RSC information to update the UI.-->but in 12th part html struture changes too when navigates to diff structure right? so both html+RSc payload needs to be fetched from server when navigation occurs right?

Exactly the right doubt 😊 — **the UI/DOM structure absolutely can change during navigation**, but the key is that **the browser does not need to receive a new HTML document** for that.

Let's separate "HTML" into two meanings

When I said:

"HTML doesn't get replaced"

I meant a new HTML document (`document`) isn't fetched and loaded.

But the DOM/HTML elements on the screen can absolutely change.

For example:

```
Current route:  
/products
```



```
DOM:  
<main>  
  <h1>Products</h1>  
  <ProductList />  
</main>
```

You click:

```
</> TypeScript
```



```
<Link href="/profile">Profile</Link>
```

The destination might be:

```
/profile
```



```
DOM:  
<main>  
  <h1>Profile</h1>  
  <ProfileDetails />  
</main>
```

So yes:

```
Products DOM  
  ↓  
React receives new RSC payload  
  ↓  
React reconciles  
  ↓  
Products elements removed  
Profile elements inserted
```



The DOM structure changes.

But do we fetch HTML + RSC payload? ❌

For a normal client-side `<Link>` navigation, generally no.

Think of two different types of navigation:

1. Full document navigation

For example:

```
HTML  
<a href="/profile">
```

when it causes a normal browser navigation:

```
Browser  
  ↓  
GET /profile  
  ↓  
Server  
  ↓  
NEW HTML document  
  ↓  
Browser replaces document
```

That's the traditional model.

2. Next.js `<Link>` client-side navigation

```
TypeScript  
<Link href="/profile">
```

Next.js can perform:

```
Browser  
  ↓  
request RSC payload for /profile  
  ↓  
Next.js server  
  ↓  
RSC Payload  
  ↓  
React  
  ↓  
Reconciliation
```

↓
DOM changes

There isn't necessarily:

Server → complete new HTML document



Instead, React already has the current document and updates its DOM.

Here's the subtle but VERY important distinction

Suppose you're here:

/products



and navigate to:

/profile



The **server** does render the new **Server Components**.

But what travels back isn't necessarily:

HTML



```
<html>
  <body>
    <main>
      ...
    </main>
  </body>
</html>
```

Instead, Next.js sends the **RSC payload**, which describes the new Server Component result/tree.

Then React uses that information to determine:

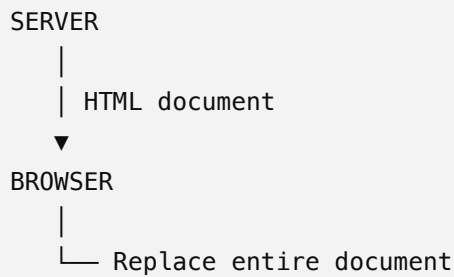
What stays?
What disappears?
What gets inserted?
What changes?



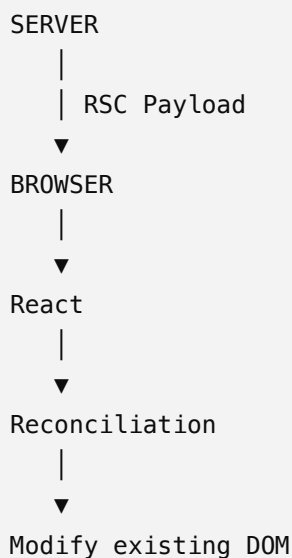
and updates the existing DOM.

Think of it like this 🧠

Full page navigation



Next.js client navigation



So:

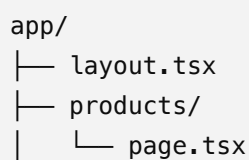
New DOM structure $\neq$ new HTML document.

That's the key.

One more wrinkle: shared layouts

This is where App Router gets really cool.

Suppose:



```
└─ profile/  
  └─ page.tsx
```

Your UI might conceptually be:

```
RootLayout  
├──  
└─ Header  
    └─ ProductsPage
```



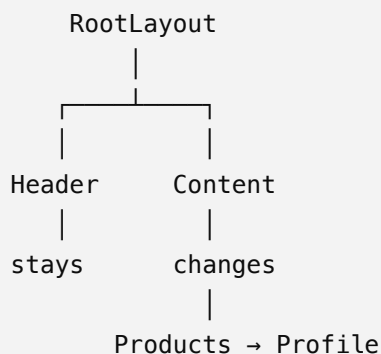
Navigate to `/profile` :

```
RootLayout  
├──  
└─ Header      ← can remain  
    └─ ProfilePage ← changes
```



So Next.js doesn't need to throw away the entire UI.

It can update the part of the tree that changed.



That's one reason the App Router's **nested layouts + RSC + client-side navigation** fit together so nicely.



So correct your Lesson 6.8 mental model to:

Initial load:

```
Request  
↓  
HTML + RSC  
↓  
Browser
```



↓
Hydration of Client Components

<Link> navigation:

Click Link
↓
RSC request
↓
New RSC payload
↓
React
↓
Reconciliation
↓
Existing DOM changes



Normal browser navigation:

Click <a>
↓
Request
↓
New HTML document
↓
Browser loads/replaces document



So your intuition that **"the HTML structure changes"** is correct. 👍

The thing to change is just:

Don't equate changing the DOM/HTML elements with downloading a new HTML document.

That's the distinction that was causing the confusion.



okay

Perfect 😊

So lock in this one-liner:

Navigation can completely change the DOM structure without downloading a new HTML document. With Next.js `<Link>`, the server can send an RSC payload, and React uses it to update the existing DOM.

That distinction will make the next caching/rendering concepts much easier. 🧠🚀



ok

👍 Perfect. We'll pick up from here whenever you're ready. 🚀



okay

😊👍 Perfect. We're all synced up.



next?

Absolutely 😊 Let's move to the next part of Lesson 6.8.

🚀 6.9 — What Actually Happens During `<Link>` Navigation?

We now know:

```
<Link>
  ↓
RSC request
  ↓
RSC Payload
  ↓
React
  ↓
```



Reconciliation



DOM changes

But there are 3 important questions left:

1. What exactly is inside the RSC payload?
2. How does React know which part of the UI to replace?
3. What happens to Client Component state during navigation?

Let's tackle them one by one.

1. What is inside the RSC payload?

Suppose you have:

```
app/  
├─ layout.tsx  
├─ products/  
│   ├─ layout.tsx  
│   └─ page.tsx  
└─ profile/  
    └─ page.tsx
```



You're currently at:

```
/products
```



Conceptually the React tree is:

```
RootLayout  
├─ Header  
└─ ProductsLayout  
    └─ ProductsPage
```



Then you click:

```
</> TypeScript  
  
<Link href="/profile">  
  Profile  
</Link>
```



The new tree is:

```
RootLayout
├── Header
└── ProfilePage
```



The RSC payload essentially communicates the **Server Component result/tree information needed to construct the new UI**.

It's not simply:

```
HTML
<h1>Profile</h1>
```



It's richer than ordinary HTML because React needs information about the component tree and boundaries.

2. How does React know what changed?

This is where **React's tree structure** matters.

Before navigation:

```
RootLayout
├── Header
└── ProductsPage
```



After navigation:

```
RootLayout
├── Header
└── ProfilePage
```



React can conceptually see:

```
RootLayout  → same
Header      → same
Content     → different
```



So the resulting DOM might go from:

```
HTML
```



```
<main>
  <h1>Products</h1>
  <div>iPhone</div>
</main>
```

to:

```
HTML

<main>
  <h1>Profile</h1>
  <div>Gopi</div>
</main>
```

The browser's DOM is updated accordingly.

Important:

React isn't taking this:

RSC Payload

and directly doing:

RSC Payload → HTML → innerHTML

Instead, the RSC result participates in React's rendering model, and React determines the UI changes.

3. What happens to Client Component state? 🔥

This is a REALLY important one.

Imagine:

```
TypeScript

"use client";

function Counter() {
  const [count, setCount] = useState(10);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

```
);  
}
```

You're currently on:

/dashboard



and the counter says:

Count: 10



You navigate somewhere else.

Whether the state survives depends on **whether that Client Component remains mounted in the resulting component tree**.

For example:

```
RootLayout  
├── Header  
│   └── Counter  
└── Page
```



Navigate:

```
/dashboard  
  ↓  
/profile
```



If **Header** and **Counter** remain part of the shared layout:

```
RootLayout  
├── Header  
│   └── Counter ← SAME  
└── ProfilePage ← changed
```



then the Client Component can remain mounted and its state can be preserved:

Count: 10



It doesn't automatically reset just because the route changed.

4. But if the Client Component belongs to the page?

Suppose:

```
/dashboard
├── DashboardPage
│   └── Counter
```



Navigate to:

```
/profile
```



Now:

```
/dashboard tree
└── removed
```



Therefore:

```
Counter
└── unmounted
```



Its state disappears.

When you return to `/dashboard` , a new Counter instance can be created:

```
count = 0
```



So:

Client Component state is tied to the lifetime/identity of that Client Component in the React tree, not simply to the URL.

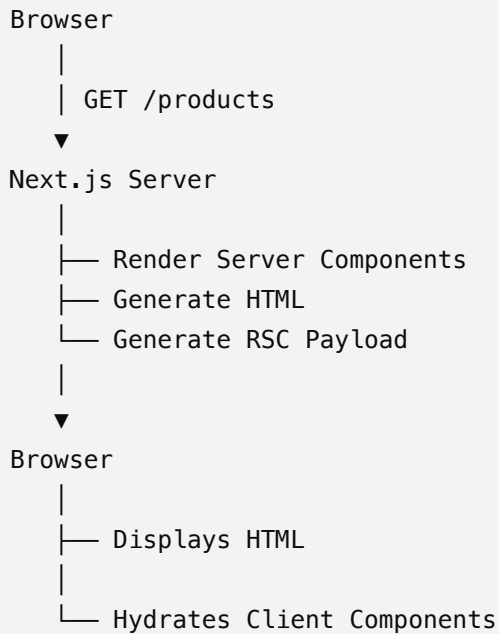


This is an excellent interview concept.

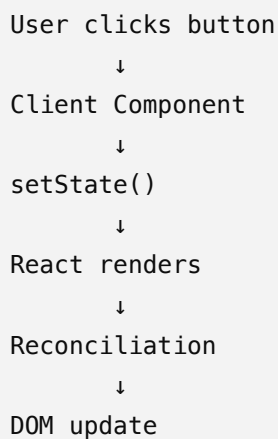
5. Now connect everything we've learned

We can finally put the entire picture together.

Initial load

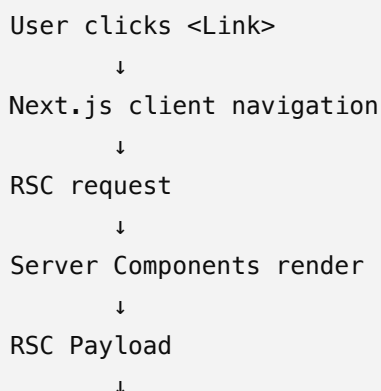


Client interaction



No server request necessarily happens.

<Link> navigation



```
Browser
  ↓
React integrates new tree
  ↓
Reconciliation
  ↓
DOM changes
```

No new full HTML document is necessarily downloaded.

`router.refresh()`

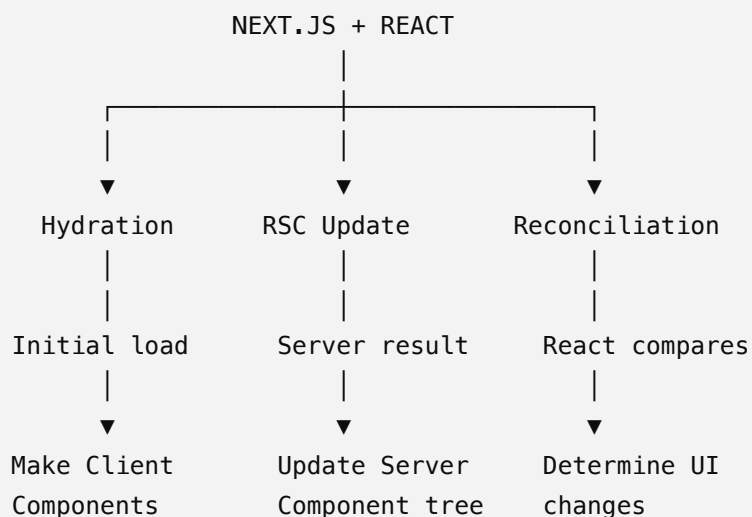
```
router.refresh()
  ↓
Server Components render again
  ↓
RSC Payload
  ↓
Browser
  ↓
React integrates updated Server Component result
  ↓
Reconciliation
  ↓
DOM update
```



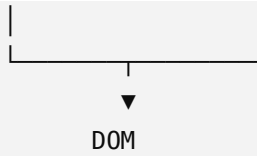
And existing Client Component state can remain when those components remain in the tree.

Your final mental model

Think of **three separate mechanisms**:



interactive



One-liner for each:

Hydration:

"Connect Client Components to existing server-rendered HTML."

RSC update:

"Bring updated Server Component results from the server."

Reconciliation:

"Determine how the React UI should change."

🎯 Next: 6.10 — RSC Payload vs HTML vs JavaScript

This is the last piece I'd do before moving forward, because we'll answer the question that usually causes the *biggest* confusion:

"If Server Component JavaScript doesn't go to the browser, but RSC payload isn't HTML either, then how does the browser actually produce the final UI?"

Once that clicks, the whole **RSC architecture becomes much less mysterious.** 😊



okay

😊 Perfect. Next we'll tackle **6.10 — RSC Payload vs HTML vs JavaScript**, especially the "if Server Component code isn't in the browser, how does the browser show it?" question.

That should tie together the whole RSC mental model nicely. 🧠🚀



[go ahead](#)

Next.js Lesson 6.10 — RSC Payload vs HTML vs JavaScript

This is **the missing puzzle piece**. Once you understand this, the earlier confusion—

"If Server Component code isn't sent to the browser, how can the browser display the Server Component?"

—becomes much easier.

Let's use one tiny example throughout.

1. Our app

```
</> TypeScript   
  
// app/page.tsx  
  
export default async function Page() {  
  const products = await getProducts();  
  
  return (  
    <main>  
      <h1>Products</h1>  
  
      <p>Count: {products.length}</p>  
  
      <Counter />  
    </main>  
  );  
}
```

And:

```
</> TypeScript   
  
// Counter.tsx  
  
"use client";  
  
export function Counter() {
```

```
const [count, setCount] = useState(0);

return (
  <button onClick={() => setCount(count + 1)}>
    Count: {count}
  </button>
);
}
```

So:

Page	→ Server Component
Counter	→ Client Component



2. First question: Where does `Page()` execute?

This:

`</>` TypeScript



```
export default async function Page() {
```

executes on the **server**.

The browser does NOT receive the source code and execute:

`</>` TypeScript



```
Page()
```

itself.

Instead:

```
Browser
  |
  | GET /
  ▼
Next.js server
  |
  | executes Page()
  ▼
React Server Components
```



The server gets:

```
products = [...]
```



and renders the Server Component tree.

3. Then how does the browser see `<h1>Products</h1>` ?

Because the server produces HTML.

Conceptually:

```
Page()  
  ↓  
React server rendering  
  ↓  
HTML
```



Something like:

```
HTML  
  
<main>  
  <h1>Products</h1>  
  <p>Count: 20</p>  
  <button>Count: 0</button>  
</main>
```

`</>`

That HTML goes to the browser.

So:

```
Server  
  |  
  | HTML  
  ▼  
Browser  
  |  
  ▼  
Paint UI
```



The browser doesn't need the Server Component source code to display the resulting HTML.

That's the first big realization. 💡

4. Then what's the RSC Payload for?

Here's where it gets interesting.

HTML answers:

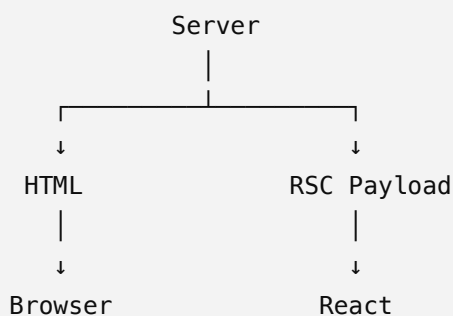
"What should the browser initially display?"

RSC Payload answers more like:

"What is the Server Component result/tree that React should work with?"

It contains React-specific information representing the Server Component tree and references to Client Components.

So conceptually:



They have **different jobs**.

5. And what about JavaScript?

Now we have three things:

HTML
RSC Payload
JavaScript

They aren't interchangeable.

HTML

Used to give the browser the initial visible UI.

```
<h1>Products</h1>
<button>Count: 0</button>
```

RSC Payload

Carries the Server Component result/tree information needed by React.

JavaScript

Provides the code necessary for **Client Components**.

For example, the browser needs the compiled JavaScript corresponding to:

```
</> TypeScript

"use client";

function Counter() {
  const [count, setCount] = useState(0);

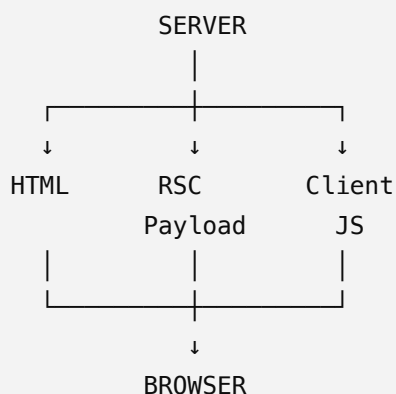
  ...
}
```

because the browser needs to execute:

```
onClick
useState
event handling
```

6. So what does the browser actually receive?

For the initial page, think approximately:



And then:

```
HTML
↓
Initial visual UI
```

```
Client JS
↓
Client Components become interactive

RSC information
↓
React understands Server Component structure/results
```

The exact transport and internal representation are more complicated than this simplified diagram, but this is the right mental model.

7. Important: Server Component code ≠ Server Component output

This distinction is **crucial**.

Suppose your server code is:

```
</> TypeScript

export default async function Page() {
  const products = await db.products.findMany();

  return (
    <div>
      <h1>Products</h1>
    </div>
  );
}
```

The browser doesn't need:

```
</> TypeScript

const products = await db.products.findMany();
```

Why?

Because that work already happened on the server.

The browser needs the **result**, not the implementation.

Think:

```
SERVER CODE
↓
execute
```

```
↓  
RESULT  
↓  
HTML / RSC representation  
↓  
BROWSER
```

This is very similar to ordering food. 😊

You don't receive the restaurant's kitchen instructions.

You receive:



Finished pizza



The kitchen's code stays in the kitchen.

8. But wait — isn't HTML itself enough?

For the **initial visual display**, HTML is extremely important.

But Next.js + React need more than just HTML because React needs to understand the application/component tree and Client Component boundaries.

That's why RSC exists.

Imagine:

HTML:

"I want you to display this."



RSC:

"Here's the React Server Component result/tree information."

Client JS:

"Here's the executable code for the interactive pieces."

Again, simplified—but useful.

9. Now let's look at navigation

This is where the difference becomes especially obvious.

Suppose:

/products



and we click:

</> TypeScript



```
<Link href="/profile">
```

The browser already has a document.

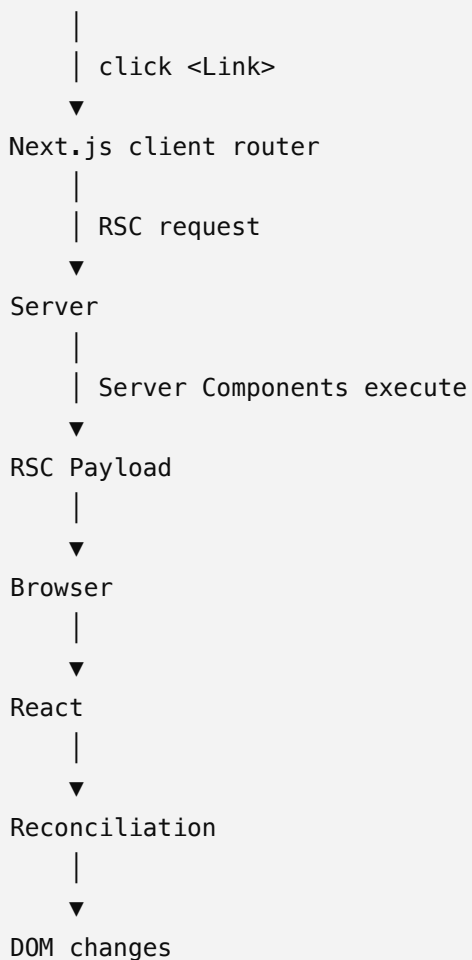
Next.js doesn't need to ask:

"Please send me an entirely new HTML document."

Instead, it can request the RSC representation for `/profile`.

Conceptually:

/products



That's why the navigation can feel much more like an application transition than a traditional document reload.

10. Does the browser turn RSC Payload directly into HTML?

This is another subtle point.

Don't think:

```
RSC Payload
  ↓
HTML string
  ↓
innerHTML
```



✗ That's not the right mental model.

Instead:

```
RSC Payload
  ↓
React understands Server Component result
  ↓
React's rendering/reconciliation process
  ↓
DOM updates
```



The browser already has a DOM.

React can update that DOM based on the new React tree.

11. `router.refresh()` makes this even clearer

Suppose your page currently shows:

```
Products: 10
```



Database changes:

```
Products: 11
```



You call:

```
</> TypeScript
router.refresh();
```



Conceptually:

```
router.refresh()
  ↓
Server executes Server Components again
  ↓
gets 11 products
  ↓
new RSC Payload
  ↓
Browser
  ↓
React
  ↓
reconciliation
  ↓
"Products: 10"
  ↓
"Products: 11"
```



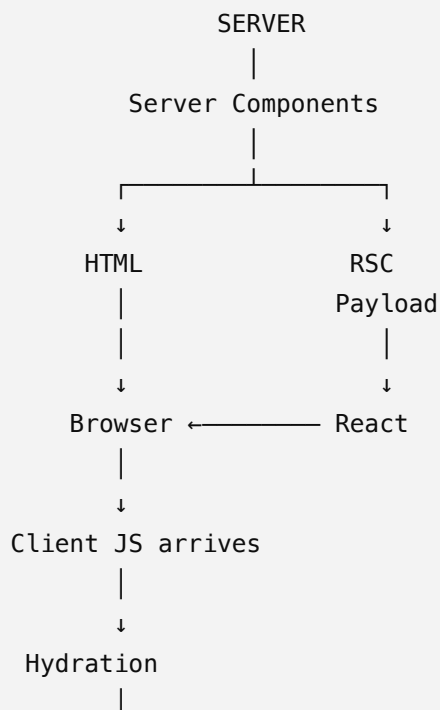
Notice:

We didn't need to download a completely new HTML document.

12. Where does hydration fit?

Now we can finally connect everything:

Initial load



↓
Client Components interactive

13. What happens to our Counter?

Initially server-generated HTML might contain:

HTML

</>

▶

📄

```
<button>Count: 0</button>
```

Browser displays it.

Then Client Component JavaScript arrives.

React knows:

"This button corresponds to Counter."

📄

Hydration connects the client-side behavior.

Now:

```
click
  ↓
  setCount(1)
  ↓
  React
  ↓
  reconciliation
  ↓
  <button>Count: 1</button>
```

📄

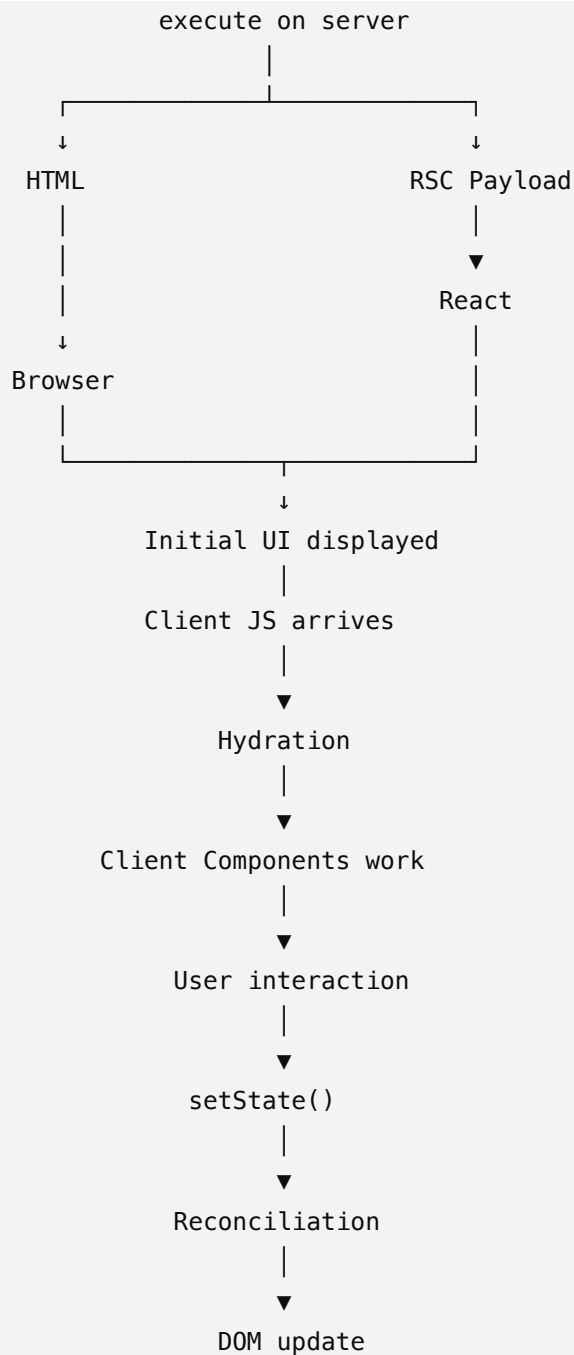
No server request required.

14. The BIG picture 🧠

Here's the mental model I'd keep:

```
NEXT.JS
  |
  ▼
Server Components
  |
```

📄



And for navigation:

```
<Link>
  ↓
RSC request
  ↓
Server Components execute
  ↓
RSC Payload
  ↓
React
  ↓
Reconciliation
  ↓
DOM changes
```



The interview answer

If they ask:

"If Server Components don't send their JavaScript to the browser, how does their UI appear?"

A clean answer:

Server Components execute on the server. Their rendered result is sent to the browser as HTML for the initial page, along with the RSC representation used by React. The browser therefore doesn't need the Server Component source code to display it. Client Component JavaScript is sent to the browser because those components need to execute there and become interactive through hydration.

And if they ask:

"What happens during client-side navigation?"

Say:

Next.js can request the RSC payload for the new route rather than downloading an entirely new HTML document. React integrates the new Server Component result into the existing tree and updates the DOM through reconciliation.



That completes the core RSC → HTML → hydration → navigation mental model.

The next logical lesson is **6.11 — "What exactly is the RSC Payload?"** We'll actually dissect a simplified payload and connect it to **Flight, Client References, Server References, serialization, and the network request** you will later inspect in DevTools.



okay



Perfect. We're ready for **6.11 — RSC Payload / React Flight** next.

That's where we'll go one level deeper into what is actually traveling over the network and how **serialization + Client References + Server References** fit together. 🚀



how many more

Not many! 😊 We’re actually getting close to the end of this **RSC/rendering deep dive**.

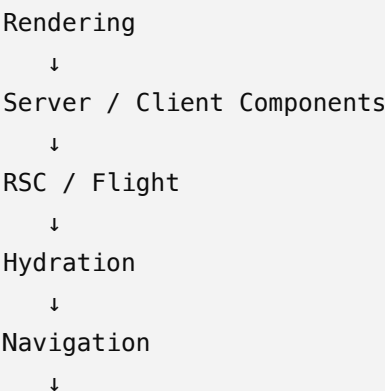
Based on the roadmap we've been following, I'd structure the remaining lessons like this:

 Remaining

Lesson	Topic	Status
6.11	RSC Payload + React Flight	 Next
6.12	Serialization, Client References & Server References	
6.13	Server ↔ Client Component boundaries	
6.14	<code>router.refresh()</code> vs navigation vs full reload	
6.15	Router Cache + RSC Payload + navigation caching	
6.16	DevTools experiment — actually inspect HTML, RSC requests, JS & Router Cache	
6.17	Complete rendering/caching mental model + interview questions	

So **~7 lessons** remain for this particular deep-dive section.

After that, we'll have covered the major Next.js architecture topics we've been targeting:



Caching



Revalidation



Router Cache



DevTools



🔥 Interview-ready mental model

And importantly, **6.16 is the hands-on one** you wanted earlier — you'll build the app first and then we'll inspect what is *actually* happening in Chrome DevTools rather than just theorizing about it. 😊

So I'd say we're in the **final stretch** of this section. 🏁



ChatGPT can make mistakes. Check important info.